# Introduction to Machine Learning

A comprehensive primer for aspiring AI engineers covering core concepts, algorithms, and practical applications.

## 1. What Is Machine Learning?

Machine learning (ML) is a subfield of artificial intelligence that gives computer systems the ability to learn from data and improve performance on tasks without being explicitly programmed. Rather than following hand-crafted rules, ML models discover patterns in data and use those patterns to make predictions or decisions.

The three main paradigms of machine learning are:

- **Supervised learning** — the model learns from labelled examples (input-output pairs).
- **Unsupervised learning** — the model finds hidden structure in unlabelled data.
- **Reinforcement learning** — an agent learns by interacting with an environment and receiving rewards or penalties.

## 2. Core Supervised Learning Algorithms

Supervised learning is the most widely used paradigm in industry. The following algorithms form the backbone of practical ML:

| Algorithm | Best For | Key Hyperparameter |
|---|---|---|
| Linear Regression | Continuous value prediction | Regularisation (L1/L2) |
| Logistic Regression | Binary classification | Regularisation strength C |
| Decision Tree | Interpretable rules | Max depth |
| Random Forest | Tabular data, high accuracy | Number of trees |
| Gradient Boosting (XGBoost) | Competitions, structured data | Learning rate |
| Support Vector Machine | High-dimensional data | Kernel, C, gamma |
| Neural Network | Images, text, audio | Architecture, learning rate |

## 3. The Machine Learning Pipeline

**Data collection.** Gather raw data from databases, APIs, web scraping, or sensors.

**Exploratory Data Analysis (EDA).** Understand distributions, missing values, outliers, and correlations using tools like pandas and matplotlib.

**Feature engineering.** Transform raw data into informative features. This step has the highest impact on model performance.

**Model selection.** Choose candidate algorithms based on data size, interpretability needs, and task type.

**Training.** Fit the model on training data by minimising a loss function using an optimiser such as gradient descent.

**Evaluation.** Measure performance on a held-out test set using metrics like accuracy, F1-score, RMSE, or AUC-ROC.

**Hyperparameter tuning.** Use grid search, random search, or Bayesian optimisation to find the best hyperparameters.

**Deployment.** Serve the model via a REST API, batch pipeline, or embedded system.

**Monitoring.** Track data drift, model degradation, and latency in production.

## 4. Overfitting and Regularisation

Overfitting occurs when a model memorises training data instead of learning generalisable patterns. It shows up as a large gap between training performance and validation performance. Common remedies include:

- Collecting more training data.
- Applying L1 (Lasso) or L2 (Ridge) regularisation to penalise large weights.
- Using dropout in neural networks.
- Early stopping — halting training when validation loss stops improving.
- Data augmentation — artificially expanding the training set.
- Reducing model complexity (fewer layers, fewer features).

## 5. Evaluation Metrics

Choosing the right metric is critical. Accuracy is misleading on imbalanced datasets. Prefer precision and recall for rare-event detection, RMSE for regression, and AUC-ROC when ranking probability outputs. The F1-score is the harmonic mean of precision and recall, useful when both matter equally.

## 6. Practical Tips for Beginners

- Start with a simple baseline before trying complex models.
- 80 percent of the value comes from data quality and feature engineering.
- Cross-validate — never evaluate on your training set.
- Use scikit-learn for classical ML and PyTorch or TensorFlow for deep learning.
- Track experiments with MLflow or Weights & Biases.
- Version your data and models as rigorously as you version code.